Name: Abhishek Kumar Singh
RollNo:2302055120002
Course: BTECH CSE B, 6TH SEM
Subject: DOT NET & C#

Assignment -2

---

a) Explain Delegates and Events in C# with a real-time example

Delegate

A **Delegate** is a **type-safe reference to a method**.
It allows methods to be **passed as parameters or stored like variables**.

Definition:
A delegate holds the reference of a method with the same signature.

Example

C#

```csharp
using System;

public delegate void Notify();   // Delegate

class Program
{
    static void Message()
    {
        Console.WriteLine("Hello from delegate");
    }

    static void Main()
    {
        Notify n = Message;   // delegate pointing to method
        n();                  // calling method through delegate
    }
}
```

Event

An **Event** is a **notification mechanism based on delegates**.
It is used when **one object notifies other objects when something happens**.

Real-time Example

Button Click Event

```csharp
using System;

public delegate void Notify();

class Button
{
    public event Notify Click;

    public void OnClick()
    {
        if (Click != null)
            Click();
    }
}

class Program
{
    static void ButtonClicked()
    {
        Console.WriteLine("Button was clicked");
    }

    static void Main()
    {
        Button b = new Button();
        b.Click += ButtonClicked;
        b.OnClick();
    }
}
```

Real-life examples

- Button click
- Email notification
- Order placed alert

---

b) What are Generics? How do they improve type safety and performance?

Generics

Generics allow creating classes, methods, and collections with a placeholder type.

Instead of specifying a data type, we use **type parameters**.

Example

```csharp
class Test<T>
{
    T data;

    public void Add(T value)
    {
        data = value;
        Console.WriteLine(data);
    }
}

class Program
{
    static void Main()
    {
        Test<int> t1 = new Test<int>();
        t1.Add(10);

        Test<string> t2 = new Test<string>();
        t2.Add("Hello");
    }
}
```

Advantages

1. Type Safety

Compiler checks the data type at **compile time**.

Example
List<int> only stores integers.

2. Performance

No **boxing and unboxing** of value types.

Example
ArrayList → boxing/unboxing
List<int> → direct storage

3. Code Reusability

Same code works for multiple data types.

---

c) Explain LINQ Architecture and write a sample query using query expression

LINQ (Language Integrated Query)

LINQ allows **querying data directly in C#** similar to SQL.

It works with:

- Collections
- Databases
- XML
- Objects

LINQ Architecture

1. LINQ Query Syntax

SQL-like syntax written in C#.

2. LINQ Providers

Convert queries to data source operations.

Examples:

- LINQ to Objects
- LINQ to SQL
- LINQ to XML

3. Data Source

Collection or database where data exists.

Example Query

```csharp
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] numbers = { 10, 20, 30, 40, 50 };

        var result = from n in numbers
                     where n > 25
                     select n;

        foreach (var r in result)
        {
            Console.WriteLine(r);
        }
    }
}
```

Output

```
30
40
50
```

---

d) Describe Multithreading in C#. How is the Thread class used?

Multithreading

Multithreading allows a program to run multiple tasks simultaneously.

Each task runs in a **separate thread**.

Advantages

- Better CPU utilization
- Faster execution
- Parallel processing

Thread Class

The **Thread class** is used to create and manage threads.

Namespace:

```
System.Threading
```

Example

```C#
using System;
using System.Threading;

class Program
{
    static void Display()
    {
        for(int i=1;i<=5;i++)
        {
            Console.WriteLine("Thread Running: " + i);
        }
    }

    static void Main()
    {
        Thread t = new Thread(Display);
        t.Start();

        Console.WriteLine("Main Thread");
    }
}
```

---

e) Explain Private Assembly and Shared Assembly (GAC) with differences

Private Assembly

A **private assembly is used by only one application**.

It is stored in the **application folder**.

Characteristics

- Used by single application
- Stored locally
- No global access

Shared Assembly (GAC)

A **shared assembly can be used by multiple applications**.

Stored in **Global Assembly Cache (GAC)**.

Characteristics

- Used by multiple applications
- Requires **strong name**
- Stored in GAC folder

Differences

Feature	**Private Assembly**	**Shared Assembly**
Usage	Single application	Multiple applications
Location	Application directory	Global Assembly Cache
Strong Name	Not required	Required
Sharing	Cannot be shared	Can be shared